

CCNA 200-301 Ağ Otomasyonu

- Otomasyona Genel Bakış
- Veri Formatları
- API (Application Programming Interface)
- REST
- Yapılandırma Yönetim Araçları

Otomasyona Genel Bakış

Otomasyonun avantajlarından bazıları:

- Makineler günde 24 saat ara vermeden çalışabilir, bu da daha fazla verim sağlar.
- Makineler daha homojen bir ürün sunmaktadır.
- Otomasyon, bir olaya veya sürece rehberlik etmeye yardımcı olabilecek bilgiler sağlamak için hızlı bir şekilde analiz edilebilen çok miktarda veri toplanmasına izin verir.
- Robotlar madencilik, yangınla mücadele ve endüstriyel kazaların temizlenmesi gibi tehlikeli koşullarda kullanılır. Bu, insanlar için riski azaltır.
- Belirli koşullar altında akıllı cihazlar enerji kullanımını azaltmak, tıbbi tanı koymak ve otomobil sürüş güvenliğini artırmak için davranışlarını değiştirebilir.
- Artık birçok cihaz, davranışlarını yönetmeye yardımcı olmak için akıllı teknoloji kullanmaktadır. Bu, yoğun talep dönemlerinde güç tüketimini azaltan akıllı bir cihaz kadar basit veya kendi kendine çalışan bir araba kadar karmaşık olabilir.
- Bir cihaz, dışarıdaki bir bilgiye dayanarak bir işlem yaptığında, söz konusu cihaz akıllı cihaz olarak adlandırılır. Etkileşimde bulunduğumuz birçok cihazın adlarında akıllı kelimesi var. Bu, cihazın ortamına bağlı olarak davranışını değiştirme yeteneğine sahip olduğunu gösterir.
- Cihazların “düşünebilmesi” için ağ otomasyon araçları kullanılarak programlanması gerekir.

Device Programmability konusu **JSON**, **XML** kavramlarını ilk kez duyanlar için karmaşık gelebilir, bu gayet doğaldır. Genel olarak cihaz programlama otomasyon sürecinin bir parçasıdır. Artık insanoğlu, cihazları **yazılımların** kontrol etmesini istemektedir. Bu bir örnek ile anlatalım. Bir misafir düşünün, bir devlet kurumuna gidiyor ve danışmaya toplantısı olduğunu söyleyerek içeriye giriyor. İçeride genelde boş olan toplantı salonuna oturuyor ki bu tür durumlarda kimse de siz kimsiniz? Kimi bekliyorsunuz? Diye sormaz. Ve bilgisayarını çıkartarak toplantı odasındaki ağ kablosunu takarak yerel ağa bağlanabiliyor. Artık bir yeri hacklemenin en zor kısmı olan dışarıdan içeri sızmayı aşmış oluyor, çünkü artık tamamen iç networkte. Artık ağ taraması yapıyor, ağdaki cihazları açık portlarına göre sınıflandırıp sunuculara erişmenin yollarını arıyor. Toplantı odasında kablolu bağlantı yoksa bu sefer araya wireless şifrenin kırılması aşaması giriyor. İşte, bu gibi durumlarda istediğimiz, yazılımın kablolu ya da kablolu bu tür normal olmayan erişimleri tespit edip, yazılımın o kişinin bağlı olduğu switch’te portu kapatması, ya da vlan’ı değiştirmesi ve tehditlerin daha oluşum aşamasında durdurulması ve tedbirlerin alınmasıdır. Bu tür sorunlar bizler için yeni değildir ve günümüzde zaten **SIEM** ya da **802.1x** gibi çözümlerle bu tür girişimler kontrol altında tutulmaya çalışılıyordu. Bu açıdan baktığımızda **Device Programmability/Automation** işlemlerinin **SIEM**’in bir parçası olacağını da söyleyebiliriz.

Tüm bunların dışında otomasyonun yararları aşağıdaki gibi sıralayabiliriz:

- Otomasyon sistemleri süreklilik sağlarlar. İnsanlar gibi yorulmak bilmez ve 7/24 aynı verimlilikle çalışabilirler.
- Otomasyon sistemleri doğru planlandığında hata da yapmazlar ve standart sonuç verirler.
- Otomasyon sistemleri insanlar için tehlikeli olan aşırı radyasyon, ısı, nem vs. içeren uygunsuz ortamda da aynı verimlilikle çalışabilirler.
- Otomasyon sistemlerinin deri dönüşleri, raporlamaları, uygulama hızları çok başarılıdır.
- Otomasyon sistemleri çok daha farklı verileri, olasılıkları hızlı bir şekilde analiz edebilir, çok fazla veriyi analiz edebilir, süreçlerde belli kriterlere göre daha hızlı kararlar verebilir.
- Otomasyon sistemlerinin tıpta, ulaşım araçlarında, hayvancılıkta ve daha bir çok alanda insanlığın rahatı ve güvenliği için de kullanılabilir.

Ağ Otomasyonu Veri Formatları

Veri formatları verileri ortak bir yapıda **tanımlamanın**, **depolamanın** ve **değiştirmenin** önemli bir yoludur. Farklı veri formatlarının olmasının sebebi farklı sistemlerin farklı komutlar kullanmasındandır. Ağ otomasyonlarında günümüzde kullanılan **en yaygın veri formatları** şunlardır:

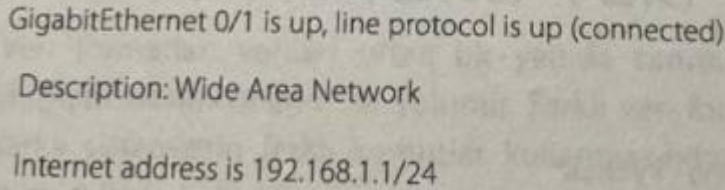
- **XML** (eXtensible Markup language)
- **JSON** (JavaScript Object Notation)
- **YAML** (YAML Ain't Markup Language)

JSON Data Format

JSON web hizmetleri ve API'ler tarafından **herkese açık veriler** sağlamak için kullanılan çok popüler bir veri biçimidir. Çok kullanılmasının en büyük sebepleri **kolay analizi** ve **Python dahil birçok programlama dilinde rahatça kullanılmasıdır**.

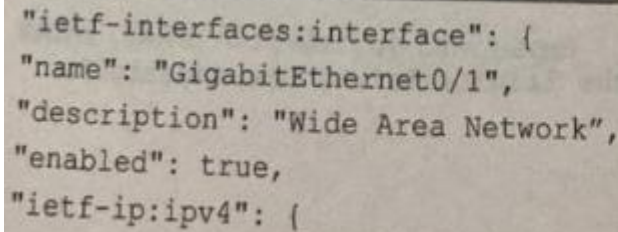
Şimdi, bu veri biçimini daha yakından tanımak için bir **örnek** verelim:

Aşağıdaki bir Router'ın interface'inin durumunu **JSON** formatında ifade edelim. Önce Router'ın interface bilgilerini bakalım:

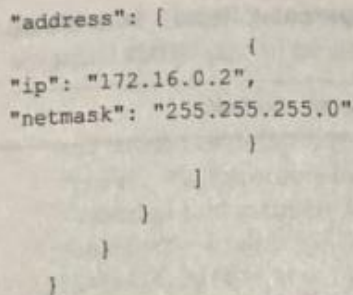


```
GigabitEthernet 0/1 is up, line protocol is up (connected)
Description: Wide Area Network
Internet address is 192.168.1.1/24
```

Şimdi bu veriyi **JSON** dilinde ifade edersek aşağıdaki gibi bir format karşımıza çıkar;



```
"ietf-interfaces:interface": {
  "name": "GigabitEthernet0/1",
  "description": "Wide Area Network",
  "enabled": true,
  "ietf-ip:ipv4": {
```



```
    "address": [
      {
        "ip": "172.16.0.2",
        "netmask": "255.255.255.0"
      }
    ]
  }
}
```

Bu yapıda her değişken ve değerin “ ile **belirlendiğine**, **en son açılan parantezin ilk kapandığında** dikkat ediniz.

JSON Veri Biçiminin Özellikleri ve Kullanım Kuralları

JSON veri formatının bazı belirgin özellikleri aşağıdakilerdir:

- Hiyerarşik, iç içe değerler için idealdir.
- Nesnelerin tanımında {} tırnaklı parantez, dizilerin tanımlarında [] köşeli parantezler kullanılır.
- Veriler key/value çiftleri olarak kullanılırlar.

JSON’da verilerin, key/value çifti olarak kullanıldığını belirtmiştik. Şimdi de bu kullanımda dikkat edilmesi gereken bazı hususlara göz atalım:

- **Keys** değerleri “ ” arasında yer alır.
- Değerler geçerli bir **JSON** veri türünde (dize, sayı, dizi vb.) olmalıdır.
- **Keys/values** kullanımlarında aralarına : konulur.
- Bir nesnenin içinde kullanılacak çok sayıda **key/values** değerleri birbirinden **virgül** ile ayrılır.
- JSON nesnelerinin tanımlanmasında **boşluğun bir zararı yoktur**.

Birden fazla değer içeren **key** değerlerine **array** (dizi) denir. **Array**’lerin kullanımı ile ilgili bazı özellikler şunlardır:

- **Array**’ler sıralı değer listelerinden oluşur.
- **Array**’lerdeki değerler birbirinden **virgül** ile ayrılır.
- Bir **array** sayı, nesne ya da **boole** gibi farklı birçok nesne içerebilir.
- Bir **key** kullanımında :’dan sonra **köşeli parantez** içinde kullanılabilecek değerler girilir.

Öğrendiklerimizi bir örnek üzerinden inceleyecek olursak;

```
{
  "addresses": [
    {
      "ip": "192.168.0.2",
      "netmask": "255.255.255.0"
    },
    {
      "ip": "192.168.0.3",
      "netmask": "255.255.255.0"
    },
    {
      "ip": "192.168.0.4",
      "netmask": "255.255.255.0"
    }
  ]
}
```

Yukarıdaki örneğimizde **IPv4** adreslerinin tanımları yer almaktadır. Buradaki öğrendiğimiz **key** değeri **addresses** kısmıdır. Her bir adres **tırnaklı parantez** ile ayrılmıştır. Her nesne ise **2 adet key/value** çifti içerir. Bunlar **ip** ve **netmask** değerleridir ve dikkat ederseniz nesne içinde kullanımlarında bunlar da birbirinden **virgül** ile ayrılmışlardır. Dikkat ederseniz **addresses** kısmındaki bu diziden hemen sonra : kullanılmış ve tüm tanım **köşeli parantez** içerisinde tanımlanmıştır.

YAML Data Format

YAML da JSON gibi verilerin **okunmasında**, **depolama** ya da **transferinde** kullanılır. YAML kullanımının bazı **özellikleri** şunlardır:

- Tırnaklı, köşeli parantez kullanımı gerekmediğinden daha **sade** bir görünümlü kullanımı vardır.
- Kullanımı JSON'a benzemekle birlikte JSON'ının **bir üst kümesi** olarak kabul edilir.
- Sade yapısından dolayı kolayca **okunabilir**, **yazılabilir**, **analiz edilebilir**.

Şimdi YAML sadeliğini bir örnekle görelim. Bir önceki konuda baktığımız aşağıdaki **JSON** kodunu bir de **YAML** olarak düzenleyelim:

```
{
  "ietf-interfaces:interface": {
    "name": "GigabitEthernet0/1",
    "description": "Wide Area Network",
    "enabled": true,
    "ietf-ip:ipv4": {
      "address": [
        {
          "ip": "192.168.0.2",
          "netmask": "255.255.255.0"
        },
        {
          "ip": "192.168.0.3",
          "netmask": "255.255.255.0"
        },
        {
          "ip": "192.168.0.4",
          "netmask": "255.255.255.0"
        }
      ]
    }
  }
}
```

Bu tanımın YAML kodu:

```
ietf-interfaces:interface:
  name: GigabitEthernet0/1
  description: Wide Area Network
  enabled: true
  ietf-ip:ipv4:
    address:
      - ip: 192.168.0.2
        netmask: 255.255.255.0
      - ip: 192.168.0.3
        netmask: 255.255.255.0
      - ip: 192.168.0.4
        netmask: 255.255.255.0
```

Görüldüğü gibi YAML ifade çok daha **sadedir**. Okuması, verileri ayrıştırması, analizi çok daha rahattır. JSON'da bir nesnedeki **key/values** değerlerinin birbirinden ayırmak için **virgül** kullanıyorduk. YAML'da ise **çift tırnak** kullanılmadan sadece **tire** kullanılır.

XML Data Format

XML formatı da JSON ya da YAML gibi verilerin okunması, depolanması ve taşınmasında uygulamalar tarafından kullanılan veri biçimidir. **XML veri biçiminin bazı özellikleri** şunlardır:

- Web sitesi tasarımlarının temeli olan HTML yapısına çok benzer bir kullanıma sahiptir.
- Analizi kolaydır. Her nesne bir **tag yapısı** içinde yer alır. <tag> DATA </tag>
- HTML'den farklı olarak XML'de **önceden tanımlanmış tag** ya da **document yapısı kullanılamaz**.

XML nesneleri de **bir** ya da **birden fazla key/value** değeri içerebilir. Her bir değer tag'ler arasında yer alır ve **açılan bir tag tekrar kapatılmalıdır**.

Örneğin: <key> VALUE </key>

Bu örnekte **VALUE** nesnesi öncesi **<key>** ile tag açılmış, değer tanımlanmış ve sonra **</key>** tag'ıyla kapatılmıştır.

Diğer veri türlerinde incelediğimiz ve aşağıda en sade YAML örneği bulunan interface IP'lerini bir de **XML kodu** olarak tanımlayalım:

Örneğimiz YAML kodu:

```
ietf-interfaces:interface:
  name: GigabitEthernet0/1
  description: Wide Area Network
  enabled: true
  ietf-ip:ipv4:
    address:
      - ip: 192.168.0.2
        netmask: 255.255.255.0
      - ip: 192.168.0.3
        netmask: 255.255.255.0
      - ip: 192.168.0.4
        netmask: 255.255.255.0
```

Bu örneğin XML kodu:

```
<?xml version="1.0" encoding="UTF-8" ?>
<ietf-interfaces:interface>
  <name>GigabitEthernet0/1</name>
  <description>Wide Area Network</description>
  <enabled>true</enabled>
  <ietf-ip:ipv4>
    <address>
      <ip>192.168.0.2</ip>
      <netmask>255.255.255.0</netmask>
    </address>
    <address>
      <ip>192.168.0.3</ip>
      <netmask>255.255.255.0</netmask>
    </address>
    <address>
      <ip>192.168.0.4</ip>
      <netmask>255.255.255.0</netmask>
    </address>
  </ietf-ip:ipv4>
</ietf-interfaces:interface>
```

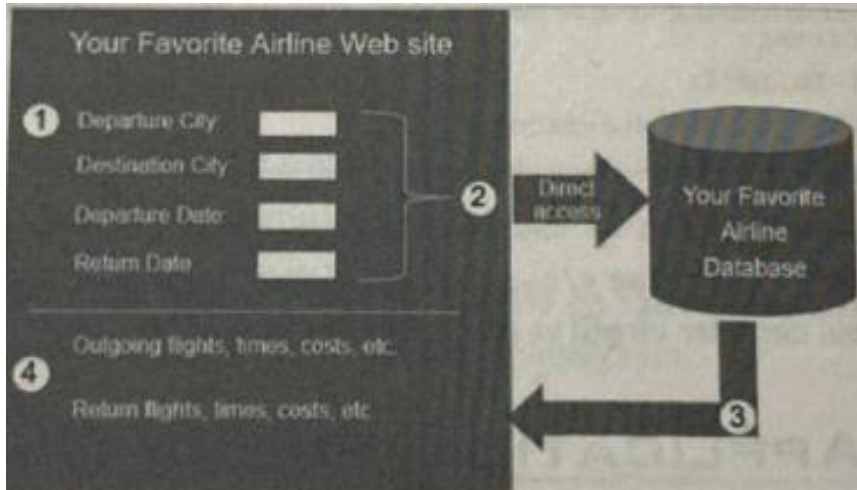
Üstteki örnekte açılan ve kapatılan tag'lere dikkat edin. Her < > arasında kalan ifade bir **tag**'dir ve </tag> ile kapatılmalıdır. JSON ve YAML'da bir nesne içerisindeki değerler **virgül** ya da **tire** değil <tag>'lar ile ayrılır.

API (Application Programming Interface)

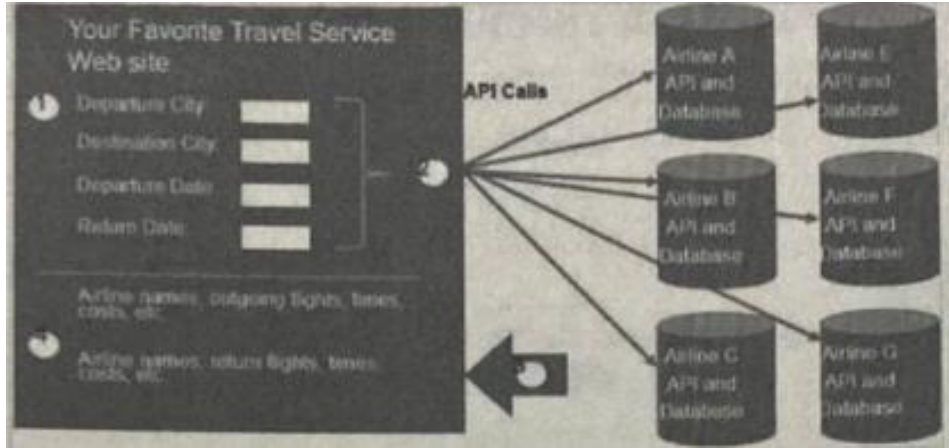
API'ler uygulamaların veri ve hizmetlere ulaşmasını sağlayan **ara katman** yazılımlarıdır. Bir uygulamanın hangi verilere ne şekilde hangi sorgularla erişebileceğini API'ler ile düzenler ve veri erişimleri, sorgulamaları ve sonuçlarını da yine API'ler aracılığıyla uygulamalara iletiriz. Aslında **API**'leri tam da **aşağıdaki şekilde** olduğu gibi mutfak ile müşteriler arasındaki iletişimi sağlayan garsona benzetebiliriz.



Şimdi bir API örneğiyle daha derinlemesine API'lere bakalım. Hava yolu firmalarının bir çoğu satışlarının büyük kısmını diğer aracı turizm, seyahat siteleri üzerinden yaparlar. Siz de böyle bir web sitesi tasarlayıp site üzerinden uçak bileti satacaksanız online olarak tarife, boş koltuk ve o uçuşun biletini alış maliyeti gibi bilgilere ihtiyacınız olacaktır. Bu bilgiler hava yolu firmasının **veri tabanında** yer aldığından ve hava yolu şirketi de kendi veri tabanını doğrudan tüm dünyaya açamayacağından dolayı bu tür ihtiyaçlar yazılan API'ler sayesinde giderilir. Web sitenize giren kullanıcılar sonuçta **aşağıdaki gibi bir arayüze** bilgileri girerler:



Bilgilerin girilip sorgunun başlatılmasıyla web siteniz API aracılığı ile güvenli bir şekilde hava yolunun **veri tabanına erişir** ve **sorgulamaları yapar**, cevapları alır ve tekrar sizin kendi veri tabanınıza **işler/işlemeyebilir** ve sorgu sonuçları ekrana getirilir. Şüphesiz iki şehir arası uçuş yapan birden fazla hava yolu şirketi olabilir. Bu durumda aynı sorgunun API'ler aracılığı ile diğer havayollarının veri tabanından da yapılması gerekir. Sonuç olarak bu isteğin diğer havayollarının veri tabanından gelen sorgu cevapları da API'lar aracılığı ile web sitesine getirilir ve ekrana sorgu sonucu olarak yansıtılır.



API Çeşitleri

Günümüzde kullanılan API'ler erişim ve yetkilerine göre **3 çeşitte** inceleyebiliriz:

Open, Internal, and Partner APIs (Açık, Dahili ve İş Ortağı API'lar)

Bir API geliştirilirken dikkat edilmesi gereken önemli bir nokta, açık, dahili ve iş ortağı API'lar arasındaki ayrımdır:

- **Open APIs or Public APIs** : Bu API'ler herkese açıktır ve herhangi bir kısıtlama olmadan kullanılabilir. Bu API'ler herkese açık olduğundan, birçok API sağlayıcısı, kullanıcının API'yı kullanmadan önce ücretsiz bir anahtar veya jeton almasını gerektirir. Bu, aldıkları ve işledikleri API isteklerinin sayısını kontrol etmeye yardımcı olmak içindir.
- **Internal or Private APIs** : Bunlar, bir kuruluş veya şirket tarafından yalnızca şirket için kullanıma yönelik verilere ve hizmetlere erişmek için kullanılan API'lardır. Dahili API örneği, yetkili satış görevlilerinin mobil cihazlarındaki satış verilerine erişmesine izin vermektir.
- **Partner APIs** : Bunlar, bir şirket ile iş ortakları veya yüklenicileri arasında, aralarındaki ticareti kolaylaştırmak için kullanılan API'lardır. Muhatap, API'yı kullanmak için bir lisansa veya başka bir izin formuna sahip olmalıdır. Havayolunun API'sını kullanan bir seyahat hizmeti, iş ortağı API'sına bir örnektir.

Web Services APIs (Web Hizmeti API'leri Türleri)

http ve https üzerinden çalıştığından entegrasyonları, kullanımı kolaydır ve yaygındır. Günümüzde birçok SMS atan program Web Service API'lerini kullanmaktadır. GSM Operatörleri kullanım kolaylığından dolayı web service API'lere hızlı destek vermektedir. Dört tür Web Service API vardır:

- Simple Object Access Protocol (SOAP)
- Representational State Transfer (REST)
- eXtensible Markup Language – Remote Procedure Call (XML-RPC)
- JavaScript Object Notation – Remote Procedure Call (JSON-RPC)

Web Service API'lerinin farklılıklarını da bir tablo üzerinden açıklayacak olursak:

Özellik	SOAP	REST (%80)	XML-RPC	JSON-RPC
Data Format (Veri Formatı)	XML	JSON, XML, YAML, vd.	XML	JSON
İlk Sürüm	1998	2000	1998	2005
Güçlülük	İyi kurulmuş	Esnek biçimlendirme ve en yaygın kullanılan	İyi kurulmuş, basitlik	Basitlik

REST and RESTful API'leri

Web tarayıcıları diğer web sitelerinden **GET/POST** yöntemleri ile **http** ve **https** protokollerini kullanarak veri alırlar, verirler. Rest API, http protokolünü kullanır. Yani üzerinden kritik verilerin geçmesi için güvenilir değildir. Rest API genelde geliştiricilerin http üzerinden basit get/post işlemleri için kullanılır.

RestFul API'ler ise aşağıdaki özelliklere sahiptir:

- **Client-Server:** Bu iletişimde hem clien kendi tarafında verileri hem de sunucu kendine gelen verileri işler. Her ikisi de karşı taraftan bağımsız olarak verileri işleyebilir.
- **Stateless:** Bu iletişimde sunucuda yalnızca verileri işlemekte kullanılır, sunucuda veriler depolanmaz ve client'ın kendi belleğinde depolanır.
- **Cacheable:** Clientlar veri işleme performansını artırmak için kendi RAM ve işlemci cache'ini kullanır.

RestFul Implementation

Restful http üzerinden web servisini kullanır. **Dört temel özelliği** vardır.

- 1- Web servisi için <http://example.com/resources> gibi temel Uniform Resource identifier (URL)'ler kullanır.
- 2- Kullanılan data formatı web servisi tarafından desteklenir. Genelde de **JSON**, **YAML** veya **XML** kullanılır ancak bu herhangi bir geçerli html standardı da olabilir.
- 3- Kullanılan uygulamalar **http** üzerinden veri aktarımı destekler.
- 4- Kullanılan API'ler de **html uyumlu** olmalıdır.

RestFul API'leri **POST**, **GET**, **PATCH**, **PUT** ve **DELETE** gibi yaygın http yöntemlerini kullanır. Aşağıdaki tabloda ise http yöntemleri ile **RestFul karşılıkları** verilmiştir:

http Uygulaması	RESTful Karşılığı
POST	Create
GET	Read
PUT/PATCH	Update
DELETE	Delete

URI, URN ve URL Tanımları

RestFul API'ler URI kullanır. URI özel bir ağ kaynağını tarif eden adrestir. Bir URI'nin iki çeşit kullanımı vardır:

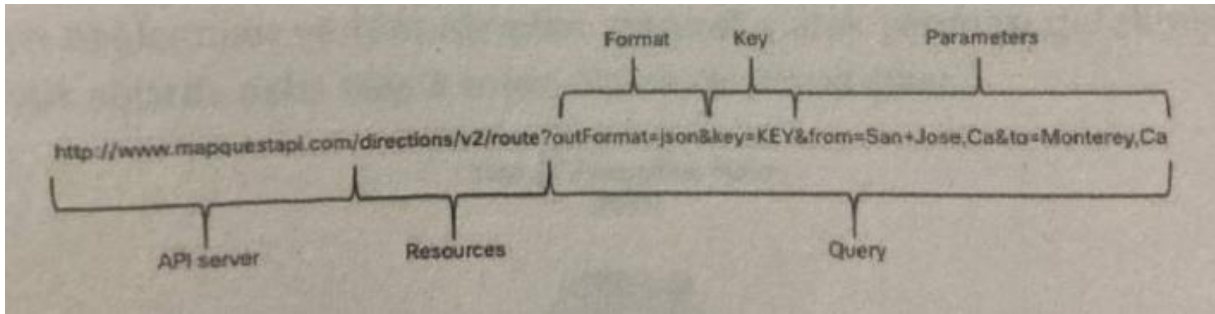
- 1- **Uniform Resource Name (URN)** – Direkt olarak kaynağın adının (web sayfası, doküman ya da resim gibi) yazılmasıyla erişilen, herhangi bir protokol kullanılmayan yöntem.
- 2- **Uniform Resource Locator (URL)** – Direkt olarak ağdaki kaynak adreslenir. http veya https kullanılarak web browser'larından erişilebilir. FTP, SFTP, SSH gibi diğer protokoller de kullanılabilir. SFTP, ftp'nin daha güvenli (secure) halidir: sftp://sftp.example.com.

Örnekte bu URI'nin parçaları <https://www.example.com/author/book.html#page155>

- Protocol/scheme : https veya ftp, sftp, mailto ve nntp gibi diğer protokoller.
- Hostname: www.example.com
- Path and file name: /author/book.html
- Fragment : #page155

RESTful İsteğin Anatomisi

RESTful Web hizmetinde, bir kaynağın URI'sine yapılan bir istek bir yanıt ortaya çıkarır. Yanıt, tipik olarak JSON biçiminde biçimlendirilmiş, ancak HTML, XML veya başka bir biçim olabilir. Şekilde MapQuest yol tarifleri API'si için URI gösterilmektedir. API talebi San Jose, California'dan Monterey, California'ya yol tarifleri içindir.



These are the different parts of the API request:

API Server – Bu, REST isteklerini yanıtlayan sunucunun URL’sidir. Bu örnekte MapQuest API sunucusudur.

Resources – İstenen API’yi belirtir. Bu örnekte MapQuest yol tarifleri API’sidir.

Query – İstemcinin API hizmetinden istediği veri biçimini ve bilgileri belirtir. Sorgular şunları içerebilir:

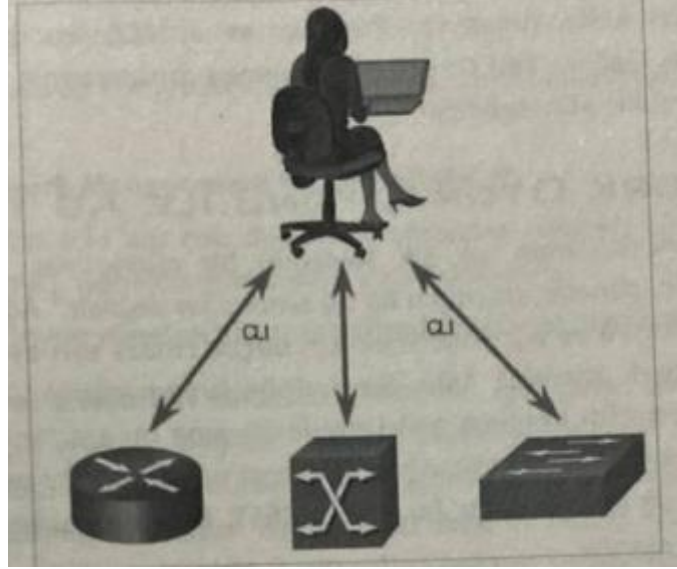
- **Format** – Genellikle JSON’dur, ancak YAML veya XML de olabilir. Bu örnekte JSON istenmiştir.
- **Key** – Genelde yetkilendirme için kullanılır. MapQuest, yol tarifleri API’si için bir anahtar gerektirir. Yukarıdaki URI’da, geçerli bir istek göndermek için “KEY” ögesini geçerli bir anahtarla değiştirmeniz gerekir.
- **Parameters** – Parametreler, istekle ilgili bilgi göndermek için kullanılır. Bu örnekte, sorgu parametreleri API’nın ihtiyaç duyduğu yönler hakkında bilgi içerir, böylece hangi yönlerin döndürüleceğini bilir: “from = San + Jose, Ca” ve “to = Monterey, Ca”.

Public yani herkese açık API’ler başta olmak üzere birçok RestFul API’ler bir **doğrulama** (key) gerektirir. Bu key’ler isteğin kaynağını belirlemek için kullanılır. Bu yüzden sizin API ile bir kaynağa erişmeden önce **karşı tarafın da doğrulayabileceği bir key’e sahip olmanız gerekir**. Genel olarak bir API’nin doğrulamaya (key) ihtiyaç duymasının sebepleri şunlardır:

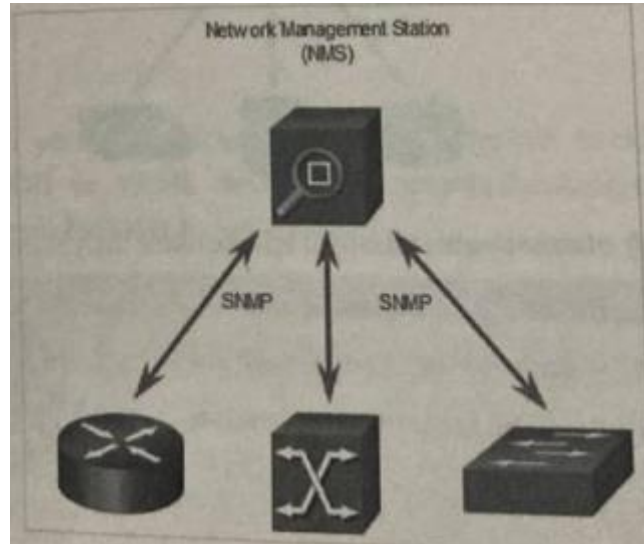
- API’yi kullanma yetkisine sahip olan kaynağın kullandığından emin olmak için doğrulama yöntemi olarak.
- API kullanan kişi sayısını belirlemek ve sınırlandırmak için.
- Kullanıcı başına istek sayısını sınırlandırmak için.
- Kullanıcılar tarafından istenen verileri kontrol etmek ve log kayıtları için.
- API kullanan kişiler hakkında bilgi toplamak için.

Yapılandırma Yönetim Araçları

Geleneksel Ağ Yöntemi



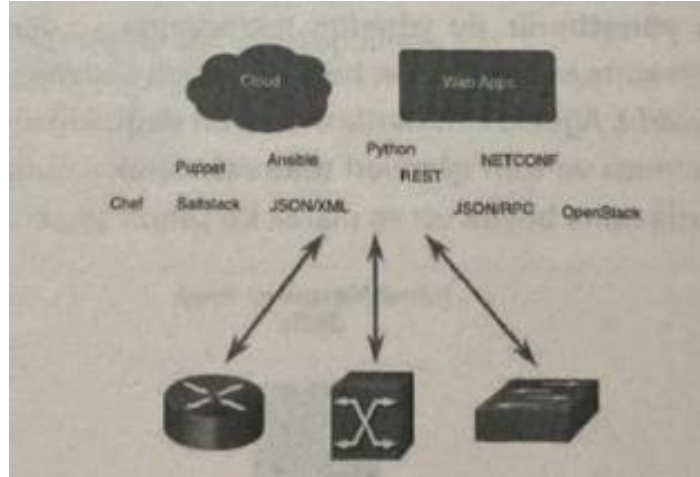
Geleneksel ağ yönteminde en çok kullanılan yönetim CLI, yani komut arayüzünden yönetimdir. Bu yönetim metodunda ağ yöneticisi her cihaza ayrı bir bağlantı kurmak zorunda ve her bağlantıda kullanıcı doğrulamasından geçmek zorundadır. Ağdaki cihazlardaki olası bir değişiklikte her cihaza tek tek, ayrı ayrı bağlanması ve tüm işlemleri **manuel** olarak yapması gerekmektedir. Bu büyük ağlarda daha büyük bir sorun olarak karşımıza çıkar.



Ağ yönetiminde **SNMP** de tercih edilmektedir. NMS dediğimiz, ağ yönetiminin merkezi ağdaki cihazlara SNMP ile bağlanmakta ve cihazdan veri alınması ve gerektiğinde de cihaza yeni ayarların gönderilmesini sağlamaktadır. Fakat SNMP her ne kadar Version 3'ü güvenli gibi olsa da **güvenlik kaygılarından dolayı çok da tercih edilmez**. Ağ cihazlarının yönetimini otomatikleştirmek için API'leri kullanabilirsiniz. Port durumları, ACL listelerinin durumlarını, sonuçlarını, **QoS** ve **Fail Over / Load Balance** durumlarının tümünü **API'ler** ile gözlemleyebilir, yönetebilirsiniz.

Ağ Otomasyonu ile Ağ Yönetimi

Ağ yöneticileri ağda ne olup bittiğini ve ağı yönetmek için birçok cihaza ayrı ayrı bağlanır ve ayrı ayrı yönetmek zorunda kalır. Bu zorluğa önlem olarak üreticiler şüphesiz **merkezi yönetim araçları** geliştirmişlerdir ama bu araçlar da yalnızca kendi ürünlerini yönetmek içindir. Ağı izlenmesi ve yönetimini otomasyon araçları ile yapmak da mümkündür. Bu araçlar **REST, Ansible, Puppet, Chef, Python, JSON, XML** olabilir.



Sağlıklı bir ağ otomasyonu yazılımı için şunlara ihtiyacımız vardır:

- 1- Cihaz yapılandırmaları için programlanabilir bir arayüz.
- 2- Konfigürasyon ve operasyonel verilerin ayrımı ve ayrı ayrı programlanabilmesi.
- 3- Entegre hata kontrolü ve kurtarma opsiyonları.

Ağ Otomasyonu açısından günümüz cihaz yönetim araçları:

- 1- Command Line Interface (CLI): CLI ağ otomasyonu için uygun ve yeterli değildir. Sebepleri:
 - a. Açık bir şekilde geri bildirim vermez.
 - b. Hata mesajları değişebilir ve netlik içermez.
 - c. Show ve debug komutları için bir standart yoktur. Üreticilerde komutlar farklıdır.
- 2- Simple Network Management Protocol (SNMP):
 - a. Ağ yönetimi için adı çok duyulmuş olmasına rağmen çok da yönetim amaçlı kullanılmaz. Genelde monitoring amaçlı kullanılır.

b. Yeterli düzeyde yönetim aracı olmaması, yazılımsal olarak yetersizliktir.

CLI, yani **komut sistemleri cihazları yönetmek** için güzel bir araçtır. Fakat otomasyon için yeterli değildir. Çünkü hata komutları net değildir. Kullandığınız komutlara ve üreticilere göre hata komutları da değişebilmektedir, bir standart kullanım yoktur. Aynı durum **show** ve **debug** komutları için de geçerlidir ve bu tür komutlar için de **standart yoktur**. Otomasyon ise cihazlara komutlar gönderir ve cihazlardan olumlu-olumsuz geri dönüşler bekler ve geri dönüşleri kendi veri tabanına işler.

Özel olarak ağ yönetimi için kullanılan araçlar:

- **Chef**
- **Ansible**
- **Puppet**
- **SaltStack**

Genel olarak bu yönetim araçları RESTful API'lerinin isteklerini karşılarlar. Hepsinin de JSON ve YAML desteği bulunmaktadır.